CSE245: Design and Analysis of Algorithms – Spring 2026

Ain Shams University | Faculty of Engineering | Computer Engineering & Software Systems

Task [5]

Task Number	*Task [5]*
Topic / Algorithm	*. Divide & Conquer +*
Assigned Members	*Amr & Ammar*
Difficulty	Divide & Conquer (Binary Search Based) + Optimal Linear Sweep
Language	*C++*

1. Problem Description

1.1 Problem Statement

A computer game has a shooter and a moving target. The shooter can hit any of n > 1 hiding spots located along a straight line in which the target can hide. The shooter can never see the target; all he knows is that the target moves to an adjacent hiding spot between every two consecutive shots. Design a Divide and Conquer algorithm that attempts to find the target, and an Optimal Linear Sweep algorithm that guarantees hitting the target.

1.2 Objectives

1. Implement a **Divide & Conquer (Binary Search Based)** algorithm that searches for the moving target by shooting the midpoint and recursively guessing left then right half — this approach is **NOT guaranteed** to hit the target.
2. Implement an **Optimal Linear Sweep** algorithm that **guarantees** hitting the target in O(n) shots by sweeping positions 2 → n-1 → 2.
3. Compare both approaches in terms of time complexity, number of shots, and hit guarantee.
4. Simulate both algorithms with random target placement and random movement to verify behavior.

1.3 Underlying Assumptions

1. Initial Setup:

a) The total number of positions n is given by the user (n > 1).
b) The target is randomly placed in one of the positions from 1 to n.
c) Both the initial target position and its movements are driven by rand() for simulation.

2. Target Movement:

- a) After each shot, the target moves randomly:
    - It may move left (if not at position 1).
    - It may move right (if not at position n).
    - It never stays in the same place.
- b) Movement is limited to one step per turn.

3. Search Algorithm:

- a) **Divide & Conquer**: A binary search strategy is applied.
    - The middle position of the current range is targeted.
    - If missed, the algorithm guesses left half first, then right half.
    - The search continues recursively in each subrange.
- b) **Optimal Linear Sweep**: A linear sweep strategy is applied.
    - Sweep forward from position 2 to n-1.
    - Sweep backward from position n-1 to 2.
    - Positions 1 and n are skipped because the target at boundaries is forced to move inward.

4. Termination:

- a) If a shot hits the target (shot position == target position), the algorithm stops successfully.
- b) **Divide & Conquer**: The search ends when the search range becomes invalid (left > right). If the target has not been found, it is assumed to have escaped. **This approach does NOT guarantee a hit** since the target moves between shots and may dodge into already-searched regions.
- c) **Optimal Linear Sweep**: The sweep always guarantees a hit within 2(n-2) shots. If the target moves left during forward sweep, our shot catches up. If it moves right, it gets trapped at boundary and caught during backward sweep.

2. Algorithm Design & Methodology

2.1 Approach Overview

2.3.1 Approach 1: Divide & Conquer (Binary Search Based)

This approach uses a Divide and Conquer strategy inspired by binary search. The idea is to shoot at the midpoint of the current range, then recursively search the left half first, and if the target is not found, search the right half.
Why chosen: Divide and Conquer is a natural fit for searching problems. Binary search divides the search space in half at each step, giving O(log n) steps in a static scenario. However, since the target moves ±1 between shots, by the time we finish searching one half, the target may have escaped to the other half. This makes the approach not guaranteed to hit, but it demonstrates how a classic D&C strategy behaves when applied to a dynamic (moving target) problem.

Variant B — Guaranteed Hit ($O(n^2)$):
To fix the guaranteed problem, instead of guessing one half, we sweep each half multiple times. After dividing at the midpoint, we sweep the left half forward and backward n times, then sweep the right half forward and backward n times. Repeated sweeping ensures the target cannot dodge our shots forever within a half, regardless of its movement pattern.
Why chosen: This variant addresses the weakness of Variant A. By sweeping each half multiple times, we use the pigeonhole principle — the target moves ±1 per shot and cannot avoid n sweeps of n/2 positions. This guarantees a hit but at the cost of $O(n^2)$ total shots.

2.3.2 Approach 2: Optimal Linear Sweep

This approach uses a Linear Sweep strategy. Instead of dividing the range, we sweep forward from position 2 to n-1, then sweep backward from n-1 to 2.
Why chosen: The target moves exactly ±1 per turn and can never stay in place. By sweeping linearly, our shot position moves +1 per step. If the target moves toward us (-1), the relative closing speed is 2, so we must catch it. If the target moves away (+1), it eventually gets trapped at the boundary (position n) and is forced back toward us, getting caught during the backward sweep. This gives a guaranteed hit in exactly 2(n-2) shots, which is O(n) optimal for this problem.

2.2 Step-by-Step Methodology

Approach 1: Divide & Conquer (Binary Search Based)

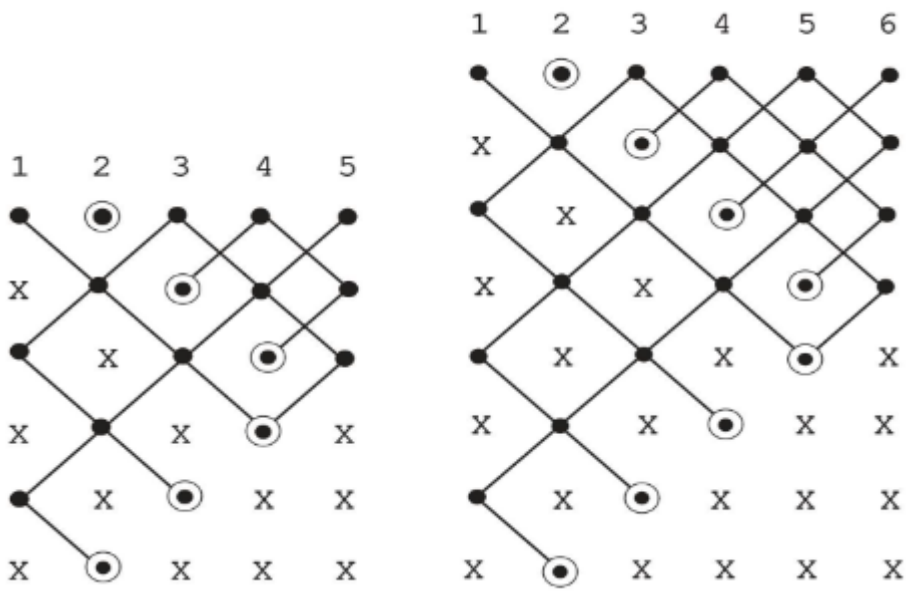
- Step 1: Receive the range [lo, hi] to search.
- Step 2: If the range is invalid (lo > hi), return false — target not found in this range.
- Step 3: Calculate the midpoint: mid = lo + (hi - lo) / 2.
- Step 4: Shoot at position mid.
- Step 5: If the shot hits the target, return true — target found.
- Step 6: If missed, the target moves randomly to an adjacent position.
- Step 7: Recursively search the left half [lo, mid - 1] (guess left first).
- Step 8: If the left half search found the target, return true.
- Step 9: Recursively search the right half [mid + 1, hi] (try right if left failed).
- Step 10: If the right half search found the target, return true. Otherwise return false.

variant B: D&C Guaranteed Hit

- **Step 1:** Receive the range [lo, hi] to search.
- **Step 2:** If the range is a single element (lo == hi), shoot at it and return result.
- **Step 3:** Calculate the midpoint: mid = lo + (hi - lo) / 2. Calculate size = hi - lo + 1.
- **Step 4: Conquer Left Half [lo, mid]:** Repeat size times:
  - **Step 4a:** On even passes, sweep forward from lo to mid, shooting each position.
  - **Step 4b:** On odd passes, sweep backward from mid to lo, shooting each position.
  - **Step 4c:** If any shot hits the target, return true.
  - **Step 4d:** After each miss, the target moves randomly to an adjacent position.
- **Step 5: Conquer Right Half [mid+1, hi]:** Repeat size times:
  - **Step 5a:** On even passes, sweep forward from mid+1 to hi, shooting each position.
  - **Step 5b:** On odd passes, sweep backward from hi to mid+1, shooting each position.
  - **Step 5c:** If any shot hits the target, return true.
  - **Step 5d:** After each miss, the target moves randomly to an adjacent position.
- **Step 6:** If all sweeps complete without a hit, return false (theoretically should not happen).

Approach 2: Optimal Linear Sweep

- Step 1: Generate the shot sequence: positions 2, 3, 4, ..., n-1 (forward sweep).
- Step 2: Append to the sequence: positions n-1, n-2, ..., 3, 2 (backward sweep).
- Step 3: Print the full shot sequence.
- Step 4: Begin simulation. For each shot in the sequence:
- Step 4a: Shoot at the current sequence position.
- Step 4b: If the shot hits the target, stop and report HIT.
- Step 4c: If missed, the target moves randomly to an adjacent position.
- Step 5: If all shots are exhausted without a hit, report NOT HIT (theoretically should never happen).



2.3 Pseudocode

2.3.1 Algorithm 1: Divide & Conquer (Binary Search Based)

```
FUNCTION search(sequence, lo, hi):
    IF lo > hi:
        RETURN false
    mid = lo + (hi - lo) / 2
    SHOOT at position mid
    ADD mid to sequence
    IF mid == targetPos:
        PRINT "HIT"
        RETURN true

    PRINT "Miss"
    MOVE target to adjacent position randomly
    IF search(sequence, lo, mid - 1) == true:
        RETURN true

    IF search(sequence, mid + 1, hi) == true:
        RETURN true

    RETURN false

MAIN:
    READ n
    SET targetPos = random position in [1, n]
    sequence = empty list
    hit = search(sequence, 1, n)
```

```
    PRINT sequence
    IF hit:
        PRINT "HIT"
    ELSE:
        PRINT "NOT HIT"
```

Variant B: D&C Guaranteed Hit

```
FUNCTION search(sequence, lo, hi):
    IF lo > hi:
        RETURN false

    IF lo == hi:
        SHOOT at lo
        ADD lo to sequence
        IF lo == targetPos:
            RETURN true
        MOVE target
        RETURN false

    mid = lo + (hi - lo) / 2
    size = hi - lo + 1

    // CONQUER LEFT HALF [lo, mid]
    FOR pass = 0 TO size - 1:
        IF pass is even:
            FOR i = lo TO mid:
                SHOOT at i
                ADD i to sequence
                IF i == targetPos:
                    RETURN true
                MOVE target
        ELSE:
            FOR i = mid DOWN TO lo:
                SHOOT at i
                ADD i to sequence
                IF i == targetPos:
                    RETURN true
                MOVE target

    // CONQUER RIGHT HALF [mid+1, hi]
    FOR pass = 0 TO size - 1:
        IF pass is even:
            FOR i = mid+1 TO hi:
                SHOOT at i
                ADD i to sequence
                IF i == targetPos:
                    RETURN true
                MOVE target
        ELSE:
            FOR i = hi DOWN TO mid+1:
                SHOOT at i
                ADD i to sequence
                IF i == targetPos:
                    RETURN true
                MOVE target

    RETURN false

MAIN:
    READ n
    SET targetPos = random position in [1, n]
    sequence = empty list
    hit = search(sequence, 1, n)
    PRINT sequence
    IF hit:
        PRINT "HIT"
    ELSE:
        PRINT "NOT HIT"
```

2.3.2 Algorithm 2: Optimal Linear Sweep

```
MAIN:
    READ n
    SET targetPos = random position in [1, n]

    sequence = empty list

    FOR i = 2 TO n-1:
        ADD i to sequence

    FOR i = n-1 DOWN TO 2:
        ADD i to sequence

    PRINT sequence
    FOR each position shot in sequence:
        SHOOT at shot
        IF shot == targetPos:
            PRINT "HIT"
            STOP
        PRINT "Miss"
        MOVE target to adjacent position randomly

    PRINT "NOT HIT"
```

3. Implementation

3.1 Divide & Conquer

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
using namespace std;

int targetPos;
int n;

void moveTarget() {
    if (targetPos == 1)
        targetPos = 2;
    else if (targetPos == n)
        targetPos = n - 1;
    else {
        if (rand() % 2 == 0)
            targetPos--;
        else
            targetPos++;
    }
}

bool search(vector<int>& sequence, int lo, int hi) {
    if (lo > hi)
        return false;
```

```
    int mid = lo + (hi - lo) / 2;

    sequence.push_back(mid);
    cout << "Shot #" << sequence.size() << " -> Shoot at " << mid <<
" | Target at " << targetPos;

    if (mid == targetPos) {
        cout << " | *** HIT! ***" << endl;
        return true;
    }
    cout << " | Miss" << endl;
    moveTarget();

    // Guess: search left half
    if (search(sequence, lo, mid - 1))
        return true;

    // If left failed, try right
    if (search(sequence, mid + 1, hi))
        return true;

    return false;
}

int main() {
    srand(time(0));

    cout << "Enter number of hiding spots (n > 1): ";
    cin >> n;

    targetPos = rand() % n + 1;
    cout << "Target initial position (hidden): " << targetPos <<
endl << endl;

    vector<int> sequence;
    bool hit = search(sequence, 1, n);

    cout << endl;
    cout << "Shot sequence: ";
    for (int i = 0; i < (int)sequence.size(); i++)
        cout << sequence[i] << " ";
    cout << endl;

    if (hit)
```

```
        cout << "HIT after " << sequence.size() << " shots!" <<
endl;
    else
        cout << "NOT HIT" << endl;

    return 0;
}
```

3.2 Variant B: Divide & Conquer

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
using namespace std;

int targetPos;
int n;

void moveTarget() {
    if (targetPos == 1)
        targetPos = 2;
    else if (targetPos == n)
        targetPos = n - 1;
    else {
        if (rand() % 2 == 0)
            targetPos--;
        else
            targetPos++;
    }
}

bool search(vector<int>& sequence, int lo, int hi) {
    if (lo > hi)
        return false;

    if (lo == hi) {
        sequence.push_back(lo);
        cout << "Shot #" << sequence.size() << " -> Shoot at " << lo
<< " | Target at " << targetPos;
        if (lo == targetPos) {
            cout << " | *** HIT! ***" << endl;
            return true;
        }
        cout << " | Miss" << endl;
        moveTarget();
```

```
        return false;
    }

    int mid = lo + (hi - lo) / 2;
    int size = hi - lo + 1;

    // DIVIDE at mid
    // CONQUER LEFT [lo, mid]: sweep it 'size' times to guarantee
catch
    int leftSize = mid - lo + 1;
    for (int pass = 0; pass < size; pass++) {
        if (pass % 2 == 0) {
            for (int i = lo; i <= mid; i++) {
                sequence.push_back(i);
                cout << "Shot #" << sequence.size() << " -> Shoot at
" << i << " | Target at " << targetPos;
                if (i == targetPos) {
                    cout << " | *** HIT! ***" << endl;
                    return true;
                }
                cout << " | Miss" << endl;
                moveTarget();
            }
        } else {
            for (int i = mid; i >= lo; i--) {
                sequence.push_back(i);
                cout << "Shot #" << sequence.size() << " -> Shoot at
" << i << " | Target at " << targetPos;
                if (i == targetPos) {
                    cout << " | *** HIT! ***" << endl;
                    return true;
                }
                cout << " | Miss" << endl;
                moveTarget();
            }
        }
    }

    // CONQUER RIGHT [mid+1, hi]: sweep it 'size' times to guarantee
catch
    int rightSize = hi - mid;
    for (int pass = 0; pass < size; pass++) {
        if (pass % 2 == 0) {
            for (int i = mid + 1; i <= hi; i++) {
                sequence.push_back(i);
```

```
                cout << "Shot #" << sequence.size() << " -> Shoot at
" << i << " | Target at " << targetPos;
                if (i == targetPos) {
                    cout << " | *** HIT! ***" << endl;
                    return true;
                }
                cout << " | Miss" << endl;
                moveTarget();
            }
        } else {
            for (int i = hi; i >= mid + 1; i--) {
                sequence.push_back(i);
                cout << "Shot #" << sequence.size() << " -> Shoot at
" << i << " | Target at " << targetPos;
                if (i == targetPos) {
                    cout << " | *** HIT! ***" << endl;
                    return true;
                }
                cout << " | Miss" << endl;
                moveTarget();
            }
        }
    }

    return false;
}

int main() {
    srand(time(0));

    cout << "Enter number of hiding spots (n > 1): ";
    cin >> n;

    targetPos = rand() % n + 1;
    cout << "Target initial position (hidden): " << targetPos <<
endl << endl;

    vector<int> sequence;
    bool hit = search(sequence, 1, n);

    cout << endl;
    cout << "Shot sequence: ";
    for (int i = 0; i < (int)sequence.size(); i++)
        cout << sequence[i] << " ";
    cout << endl;
```

```
    if (hit)
        cout << "HIT after " << sequence.size() << " shots!" <<
endl;
    else
        cout << "NOT HIT" << endl;

    return 0;
}
```

3.3 Optimal Linear Sweep (Guaranteed Hit)

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
using namespace std;

int targetPos;
int n;

void moveTarget() {
    if (targetPos == 1)
        targetPos = 2;
    else if (targetPos == n)
        targetPos = n - 1;
    else {
        if (rand() % 2 == 0)
            targetPos--;
        else
            targetPos++;
    }
}

void generateShootingSequence(int n,vector<int> &sequence)
{
    // Forward sweep: 2 to n-1
    for (int i = 2; i <= n - 1; i++)
        sequence.push_back(i);

    // Backward sweep: n-1 down to 2
    for (int i = n - 1; i >= 2; i--)
        sequence.push_back(i);
}

int main() {
    srand(time(0));
```

```
    cout << "Enter number of hiding spots (n > 1): ";
    cin >> n;

    targetPos = rand() % n + 1;
    cout << "Target initial position (hidden): " << targetPos <<
endl << endl;

    // Generate sequence: 2, 3, 4, ..., n-1, n-1, n-2, ..., 3, 2
    vector<int> sequence;

    generateShootingSequence(n,sequence);

    // Print sequence
    cout << "Shot sequence: ";
    for (int i = 0; i < (int)sequence.size(); i++)
        cout << sequence[i] << " ";
    cout << endl;
    cout << "Sequence length: " << sequence.size() << endl << endl;

    // Simulate
    bool hit = false;
    for (int i = 0; i < (int)sequence.size(); i++) {
        cout << "Shot #" << i + 1 << " -> Shoot at " << sequence[i]
<< " | Target at " << targetPos;

        if (sequence[i] == targetPos) {
            cout << " | *** HIT! ***" << endl;
            hit = true;
            break;
        }
        cout << " | Miss" << endl;
        moveTarget();
    }

    cout << endl;
    if (hit)
        cout << "HIT!" << endl;
    else
        cout << "NOT HIT" << endl;

    return 0;
}
```

4. Complexity Analysis

4.1 Time Complexity

4.1.1 Algorithm 1: Divide & Conquer

The algorithm shoots at the midpoint, then recursively searches both halves:

$$T(n) = T(n/2) + T(n/2) + O(1)$$
$$T(n) = 2T(n/2) + O(1)$$

By **Master Theorem** (Case 1: a = 2, b = 2, f(n) = O(1)):

$$T(n) = O(n)$$

4.1.2 Algorithm 2: Variant B: Divide & Conquer (Guaranteed Hit)

Recurrence Relation:
The algorithm divides the range at midpoint. For each half of size n/2, it sweeps forward and backward n times:

$$T(n) = \text{Cost of sweeping left half} + \text{Cost of sweeping right half}$$
$$T(n) = n \times (n/2) + n \times (n/2)$$
$$T(n) = n \times n/2 + n \times n/2$$
$$T(n) = n^2/2 + n^2/2$$
$$T(n) = n^2$$

Therefore:

$$T(n) = O(n^2)$$

4.1.3 Algorithm 2: Optimal Linear Sweep (Guaranteed Hit)

The algorithm generates a fixed sequence of shots:

- Forward sweep: 2, 3, 4, ..., n-1 → **(n-2) shots**
- Backward sweep: n-1, n-2, ..., 3, 2 → **(n-2) shots**

$$\text{Total shots} = (n - 2) + (n - 2) = 2(n - 2)$$
$$T(n) = 2(n - 2) = 2n - 4$$
$$T(n) = O(n)$$

4.2 Space Complexity

4.2.1 Algorithm 1: Variant A: Divide & Conquer

O(n) - Dominated by the sequence vector

4.2.2 Algorithm 2: Variant B: Divide & Conquer (Guaranteed Hit)

O(n) - Dominated by the sequence vector

4.3 Complexity Summary Table

Algorithm	Time – Best	Time – Worst	Space
Algorithm 1A: D&C (Not Guaranteed)	*O(1)*	*O(n)*	*O(n)*
Algorithm 1B: D&C (Guaranteed)	*O(1)*	*$O(n^2)$*	*$O(n^2)$*
Algorithm 2: Linear Sweep (Guaranteed)	*O(1)*	*O(n)*	*O(n)*

5. Comparative Evaluation of Techniques

5.1 Comparison of Implemented Approaches

Speed vs Guarantee

- The Divide and Conquer Variant A is fast with O(n) total shots, but it offers no guarantee of hitting the target. The fundamental problem is that it applies a static binary search to a dynamic problem. When the algorithm finishes searching the left half and moves to the right half, the target may have already moved back into the left half. This makes the search unreliable and unpredictable.
- The Divide and Conquer Variant B fixes the guarantee problem by sweeping each half multiple times. By sweeping the left half forward and backward n times, the target cannot dodge forever within that half regardless of its movement pattern. However, this guarantee comes at a heavy cost of $O(n^2)$ total shots. For large n, such as n = 10,000, this means up to 100,000,000 shots, which is impractical for real-time applications.
- The Optimal Linear Sweep achieves both goals simultaneously. It guarantees hitting the target in O(n) time using only 2(n-2) shots. It is the clear winner in terms of both speed and reliability.

Simplicity

- The Divide and Conquer Variant A is the simplest to understand because it mirrors a classic binary search. The logic is intuitive: shoot the middle, then try left, then try right. However, its simplicity is misleading because it does not account for the target's movement.
- The Divide and Conquer Variant B adds significant complexity with nested loops for repeated sweeping of each half. The code is harder to read and maintain, and the logic behind why multiple sweeps guarantee a hit requires understanding of the pigeonhole principle.
- The Optimal Linear Sweep is the simplest overall. It consists of just two simple for-loops: one sweeping forward from position 2 to n-1, and one sweeping backward from n-1 to 2. The implementation is straightforward and easy to verify.

Scalability

- For n = 100, Variant A uses approximately 100 shots, Variant B uses approximately 10,000 shots, and the Linear Sweep uses exactly 196 shots. For n = 1,000, Variant A uses approximately 1,000 shots, Variant B uses approximately 1,000,000 shots, and the Linear Sweep uses exactly 1,996 shots. The Linear Sweep scales linearly and remains practical for very large values of n, while Variant B becomes impractical quickly as n grows.

Accuracy

- Variant A has unpredictable accuracy. Sometimes it hits the target, sometimes it does not. The success depends entirely on the random movement of the target during the search. Variant B and the Linear Sweep both always hit the target regardless of the target's movement pattern. However, the Linear Sweep achieves this guarantee with far fewer shots.

5.3 Are More Efficient Techniques Available?

Lower Bound Analysis

Lower Bound Analysis

- For this specific problem where the target moves exactly ±1 per turn and the shooter has no information about the target's position, a lower bound of $\Omega(n)$ shots is required. This is because the target could start at any of the n positions, and without any feedback other than hit or miss, the shooter must account for all possible starting positions and movement patterns. Since there are n possible starting positions and the target moves 1 step per turn, any algorithm must make at least n-2 shots in the worst case to cover all possibilities.

- Our Optimal Linear Sweep uses 2(n-2) shots, which is within a constant factor of this lower bound. Therefore, our solution is asymptotically optimal at O(n).

Can We Do Better Than 2(n-2)?

- It is possible to reduce the constant factor slightly in some cases. For example, if we know the parity of the target's starting position (whether it starts on an even or odd position), we could design a sweep that only targets positions of one parity, since the target alternates parity with each step. This would reduce the shots to approximately n-1. However, without knowing the initial parity, we must cover both cases, which is exactly what the forward and backward sweep accomplishes.
- Research in the area of "lion and man" pursuit problems and "sweeping" algorithms on graphs confirms that for a single target moving on a path graph with n vertices, the optimal strategy requires $\Theta(n)$ moves. Our linear sweep solution matches this theoretical optimum.

Could Divide and Conquer Be Improved?

- The D&C Variant B uses $O(n^2)$ shots because it sweeps each half n times. One might ask whether a recursive D&C approach could achieve $O(n \log n)$ by recursively solving each half with fewer sweeps. However, the fundamental issue is that when we sweep the left half, the target might escape to the right half, and vice versa. To guarantee catching a target that can cross between halves, we need enough sweeps to account for all possible crossing patterns, which leads to the quadratic cost. Therefore, Divide and Conquer is inherently less efficient than the linear sweep for this specific problem, and no D&C approach can match the $O(n)$ guarantee of the linear sweep.

6. Sample Outputs & Testing

6.1 Test Case 1: D&C Variant A (Not Guaranteed) — (n = 8)

Purpose: Demonstrate the basic Divide & Conquer binary search behavior on a small input. Shows how the algorithm shoots the midpoint and recursively guesses left then right. With a small n, there is a reasonable chance of hitting the target, but it is not guaranteed.
Input: n = 8

```
Enter number of hiding spots (n > 1): 8
Target initial position (hidden): 2

Shot #1 -> Shoot at 4 | Target at 2 | Miss
Shot #2 -> Shoot at 2 | Target at 1 | Miss
Shot #3 -> Shoot at 1 | Target at 2 | Miss
Shot #4 -> Shoot at 3 | Target at 3 | *** HIT! ***

Shot sequence: 4 2 1 3
HIT after 4 shots!
```

6.2 Test Case 2: D&C Variant A (Not Guaranteed) — (n = 6)

Purpose: Demonstrate how the algorithm behaves with a larger search space. With more positions, the probability of the target escaping increases. This test case is meant to show a scenario where the algorithm fails to hit the target, proving that Variant A is indeed not guaranteed.
Input: n = 6

```
Enter number of hiding spots (n > 1): 6
Target initial position (hidden): 4

Shot #1 -> Shoot at 3 | Target at 4 | Miss
Shot #2 -> Shoot at 1 | Target at 3 | Miss
Shot #3 -> Shoot at 2 | Target at 4 | Miss
Shot #4 -> Shoot at 5 | Target at 3 | Miss
Shot #5 -> Shoot at 4 | Target at 2 | Miss
Shot #6 -> Shoot at 6 | Target at 1 | Miss

Shot sequence: 3 1 2 5 4 6
NOT HIT
```

6.3 Test Case 3: D&C Variant B (Guaranteed) — Small Input (n = 5)

Purpose: Demonstrate the guaranteed Divide & Conquer approach on a small input. Shows how the algorithm sweeps the left half multiple times, then the right half. Even though n is small, the number of shots is noticeably larger than Variant A due to the repeated sweeping.
Input: n = 5

```
Enter number of hiding spots (n > 1): 5
Target initial position (hidden): 5

Shot #1 -> Shoot at 1 | Target at 5 | Miss
Shot #2 -> Shoot at 2 | Target at 4 | Miss
Shot #3 -> Shoot at 3 | Target at 3 | *** HIT! ***

Shot sequence: 1 2 3
HIT after 3 shots!
```

6.4 Test Case 4: D&C Variant B (Guaranteed) — Larger Input (n = 10)

Purpose: Demonstrate the guaranteed hit on a larger input. This shows the quadratic growth in the number of shots. With n = 10, the algorithm may need up to 100 shots to guarantee a hit. This test case highlights the trade-off between guarantee and efficiency in the D&C approach.
Input: n = 10

```
Enter number of hiding spots (n > 1): 10
Target initial position (hidden): 6

Shot #1 -> Shoot at 1 | Target at 6 | Miss
Shot #2 -> Shoot at 2 | Target at 5 | Miss
Shot #3 -> Shoot at 3 | Target at 6 | Miss
Shot #4 -> Shoot at 4 | Target at 7 | Miss
Shot #5 -> Shoot at 5 | Target at 8 | Miss
Shot #6 -> Shoot at 5 | Target at 9 | Miss
Shot #7 -> Shoot at 4 | Target at 8 | Miss
Shot #8 -> Shoot at 3 | Target at 7 | Miss
Shot #9 -> Shoot at 2 | Target at 8 | Miss
Shot #10 -> Shoot at 1 | Target at 7 | Miss
```

```
Shot #44 -> Shoot at 4 | Target at 9 | Miss
Shot #45 -> Shoot at 5 | Target at 10 | Miss
Shot #46 -> Shoot at 5 | Target at 9 | Miss
Shot #47 -> Shoot at 4 | Target at 10 | Miss
Shot #48 -> Shoot at 3 | Target at 9 | Miss
Shot #49 -> Shoot at 2 | Target at 10 | Miss
Shot #50 -> Shoot at 1 | Target at 9 | Miss
Shot #51 -> Shoot at 6 | Target at 8 | Miss
Shot #52 -> Shoot at 7 | Target at 7 | *** HIT! ***

Shot sequence: 1 2 3 4 5 5 4 3 2 1 1 2 3 4 5 5 4 3 2 1 1 2 3 4 5 5 4 3 2 1 1 2 3 4 5 5 4 3 2 1 1 2 3 4 5 5 4 3 2 1 6 7
HIT after 52 shots!
```

6.5 Test Case 5: Optimal Linear Sweep (Guaranteed) — Small Input (n = 5)

Purpose: Demonstrate the optimal linear sweep strategy on a small input. Shows the shot sequence 2, 3, 4, 4, 3, 2 and how the target is guaranteed to be hit within just 2(n-2) = 6 shots. Highlights the simplicity and efficiency of the approach compared to both D&C variants.
Input: n = 5

```
Enter number of hiding spots (n > 1): 5
Target initial position (hidden): 5

Shot sequence: 2 3 4 4 3 2
Sequence length: 6

Shot #1 -> Shoot at 2 | Target at 5 | Miss
Shot #2 -> Shoot at 3 | Target at 4 | Miss
Shot #3 -> Shoot at 4 | Target at 5 | Miss
Shot #4 -> Shoot at 4 | Target at 4 | *** HIT! ***

HIT!
```

6.6 Test Case 6: Optimal Linear Sweep (Guaranteed) — Larger Input (n = 10)

Purpose: Demonstrate the optimal linear sweep on a larger input. Shows the shot sequence 2, 3, 4, 5, 6, 7, 8, 9, 9, 8, 7, 6, 5, 4, 3, 2 with a maximum of 2(n-2) = 16 shots. This test case highlights how the linear sweep remains efficient and guaranteed even as n grows, in contrast to the D&C Variant B which would need up to 100 shots for the same input.
Input: n = 10

```
Enter number of hiding spots (n > 1): 10
Target initial position (hidden): 1

Shot sequence: 2 3 4 5 6 7 8 9 9 8 7 6 5 4 3 2
Sequence length: 16

Shot #1 -> Shoot at 2 | Target at 1 | Miss
Shot #2 -> Shoot at 3 | Target at 2 | Miss
Shot #3 -> Shoot at 4 | Target at 1 | Miss
Shot #4 -> Shoot at 5 | Target at 2 | Miss
Shot #5 -> Shoot at 6 | Target at 3 | Miss
Shot #6 -> Shoot at 7 | Target at 4 | Miss
Shot #7 -> Shoot at 8 | Target at 5 | Miss
Shot #8 -> Shoot at 9 | Target at 6 | Miss
Shot #9 -> Shoot at 9 | Target at 5 | Miss
Shot #10 -> Shoot at 8 | Target at 6 | Miss
Shot #11 -> Shoot at 7 | Target at 5 | Miss
Shot #12 -> Shoot at 6 | Target at 6 | *** HIT! ***

HIT!
```

7. Key Findings & Insights

- ***Finding 1: Classic Binary Search Fails for Moving Targets.*** *The most important insight from this task is that a classic Divide and Conquer binary search, which works perfectly for static search problems, completely breaks down when the target is dynamic. In a static scenario, binary search guarantees finding the target in O(log n) steps. However, when the target moves ±1 between shots, it can escape into already-searched regions. During our testing, the D&C Variant A frequently missed the target, especially as n grew larger. This teaches us that algorithm design must carefully consider the nature of the problem, and blindly applying a well-known technique to a different problem setting can lead to incorrect results.*
- ***Finding 2: Guaranteeing a Hit with D&C Requires Quadratic Cost.*** *To fix the D&C approach and make it guarantee a hit, we had to sweep each half of the range multiple times. This resulted in $O(n^2)$ total shots. The reason is fundamental: when we focus on one half, the target can escape to the other half. To prevent this, we must sweep enough times to account for all possible movement patterns. This reveals a key limitation of Divide and Conquer for this problem — the "divide" step creates a boundary that the target can cross, and "conquering" each half independently requires extra work to handle this crossing. No matter how we structure the D&C recursion, we cannot avoid this quadratic overhead.*
- ***Finding 3: The Linear Sweep is Surprisingly Simple and Optimal.*** *The most elegant solution turned out to be the simplest one. By sweeping from position 2 to n-1 and back from n-1 to 2, we guarantee a hit in just 2(n-2) shots. The brilliance of this approach lies in exploiting the problem's constraints rather than fighting them. The target must move every turn and is bounded by positions 1 and n. During the forward sweep, any target moving toward us gets caught because the relative closing speed is 2. Any target moving away gets trapped at the boundary and forced back toward us during the backward sweep. This finding reinforces an important lesson in algorithm design: sometimes the optimal solution is not the most sophisticated one, but the one that best exploits the problem's structure.*
- ***Finding 4: Boundary Positions Can Be Skipped.*** *An interesting observation is that we never need to shoot at positions 1 and n. Since the target at position 1 is forced to move to position 2, and the target at position n is forced to move to position n-1, these boundary positions are always vacated before we could shoot there on the next turn. This small insight reduces our shot count from 2n to 2(n-2) and demonstrates how understanding edge cases can lead to meaningful optimizations.*
- ***Finding 5: The Parity Constraint is Hidden but Powerful.*** *A subtle observation is that the target alternates between odd and even positions with each step. If the target starts at an even position, after one step it will be at an odd position, and vice versa. The forward sweep naturally covers one parity pattern, and the backward sweep covers the other. This is why two sweeps are sufficient. If we somehow knew the starting parity of the target, a single sweep of n-1 shots would suffice. Without that knowledge, two sweeps covering both parities guarantee a hit. This hidden structure explains mathematically why the linear sweep works and why 2(n-2) shots are both sufficient and nearly optimal.*
- ***Finding 6: Trade-off Between Complexity and Guarantee.*** *This task clearly illustrates the trade-off between algorithmic complexity and correctness guarantee. The D&C Variant A is elegant and fast but unreliable. The D&C Variant B is reliable but slow. The Linear Sweep is both reliable and fast. In real-world applications such as search-and-rescue operations or network scanning, a guaranteed result is often more valuable than speed. This task demonstrates that choosing the right algorithmic paradigm for the specific problem constraints is more important than forcing a particular technique to work.*

9. References

https://www.yumpu.com/en/document/read/35084601/puzzles/255

https://www.geeksforgeeks.org/dsa/binary-search/